

Lesson 9, Part 2 Activity: Estimate One Slice of Your Project

CS 5388 / CS 4390 Software Project Management | Team activity, complete together

Team: 10 Project: DineCheck (Campus Dining System)

What this activity is for

Your Sprint 2 business case has to answer a question every sponsor asks: how much work would this take, and what would it cost? You are not going to build the system in this course. You are doing what a project manager does before a project is approved: estimating the work so a decision can be made.

Estimating a whole project at once hides mistakes. So you will start small. Pick one or two major parts of your project, called a slice, and estimate only that slice. A small slice lets you count everything, check every number, and see clearly where your assumptions matter.

You will estimate the same slice twice, using two methods that start from different information. Then you will compare the two answers, explain why they differ, and present a range with your assumptions written down. That range becomes the Estimation Appendix on your Sprint 2 portal page.

What to bring

1. Your Release 1 capability list (Step 1 of the Part 1 starter).
2. Your backlog from the Part 1 activity: user stories, MoSCoW categories, and planning poker point sizes.
3. Your similar-products research from Part 1 (small and big products, with timelines and team sizes).
4. The Part 2 slides, especially Slides 13 to 17 (sizing), 19 to 24 (team size), 25 to 27 (story points), and 31 to 33 (reconciling).

How the activity fits together

Step	What you do	What you end up with
1	Pick your slice	One or two major parts of your project, with their stories, screens, and data listed
2	Choose two estimation methods	Two methods that use different starting information
3	Estimate the slice with each method	Two effort estimates, both in team hours
4	Check the estimates for team size	Confidence that coordination costs are included
5	Compare the two estimates	A written explanation of why they differ
6	Present the result	A dated range, in hours, dollars, and weeks, ready for the business case

Key terms

Term	What it means in this activity
Slice	One or two major parts of your project that a user would recognize, such as “buy a permit.” You estimate only this.
Size	How much software the slice contains, measured in function points, use-case points, or story points.
Effort	Total team hours of work. If 5 people each work 10 hours, that is 50 hours of effort.
Productivity rate	Hours of effort per unit of size, such as 8 hours per function point. The rate drives the whole estimate.
Assumption	Any number you chose rather than measured. Label it, so a reader knows it could change.
Velocity	Story points a team completes per sprint. You have no sprint history yet, so yours is an assumption.
Loaded rate	The full hourly cost of one person: salary plus benefits, overhead, and tools.
Range	A low and a high estimate. Honest estimates are ranges, not single numbers.
O, M, P	Optimistic, most likely, and pessimistic estimates, used in the PERT formula $(O + 4M + P) \div 6$.

About the examples

Every italic example uses the same slice from the fictional Campus Parking System: “Buy a semester permit.” Following one slice through every method shows how the pieces connect. Your own numbers come from your project.

Step 1. Pick your slice

A good slice is a part of your project that a user would recognize and that is useful on its own. It usually covers 3 to 8 user stories from your backlog.

How to choose

1. Start with a Must-have capability that is central to your project. If it did not exist, the project would not make sense.
2. If you pick a second part, make it different in kind, such as one part users interact with and one part that produces reports or connects to another system.
3. Do not pick the easiest part. The slice should include some real complexity, such as payments, another system, or stored data.
4. Write down what is out of the slice, so your estimate has a clear boundary.

Question	Our slice
Slice name (one or two major parts)	Student visibility of campus dining options, locations, and sold-out items within each menu + mobile visibility, plus the staff update form that keeps sold-out status current.
Why we chose it	One of the core problems we are attempting to solve is a one-stop solution to finding campus dining availability before engaging pointless commute for students.
Screens or forms where users enter data	Staff update form (location hours, menu items, sold-out status). Students choose dietary filters on the location list and menu view.
Reports, results, or lookups users see	Location list (open dining options) and menu view (available/sold-out items for each menu).
Data the slice stores	Per location times, menus and dining option availability.
Other systems it connects to	N/A. Sold-out status comes from dining staff through the staff update form, not from a point-of-sale system.
Out of the slice (not included)	Students can view a current crowd level and estimate wait time at each location.

Stories in the slice

User story	MoSCoW	Points
As a student, I want to view campus dining locations so that I can see what dining options are available	Must	3
As a student, I want to view each dining location's hours and open/closed status so that I know when I can visit.	Must	3
As a student, I want to view menu items for each dining location so that I can decide where to eat.	Must	3
As a student, I want to see whether menu items are available or sold out so that I do not waste time going somewhere for an unavailable item.	Must	5

User story	MoSCoW	Points
As a student, I want to filter dining options based on dietary restrictions or preferences so that I can find suitable food.	Should	5
As a student, I want the dining information to work well on my phone so that I can quickly check dining options while on campus	Should	3
As dining staff, I want to mark an item sold out or back in stock so that students see the change before they walk over.	Must	3
Total points		25

Example slice, “Buy a semester permit”: a student registers a vehicle (3 points), chooses a lot and permit type (5), pays online (8), receives a digital permit by email (3), and looks up the permit later (2), for 21 points. It stores permit and vehicle records and connects to the student registry and a payment service. Citations and appeals are out of the slice.

Step 2. Choose two methods

Each method starts from different information. Choose two that do not share the same starting point, so that if they agree, the agreement means something (Part 2, Slide 31).

Method	It starts from	A good choice if	Watch out for
Analogous	Similar products and how long they took	You found at least one product close to yours	Copying a big company's timeline without adjusting
Function points	Counts of what users enter, see, and store	Your Step 1 table lists screens, results, and data	A productivity rate with no source
Use-case points	The actors and use cases in your slice	You can list who uses the slice and for what	Default adjustment factors used without saying so
Story points to cost	Your planning poker points and a velocity	Every story in the slice has points	Velocity is a guess until you run sprints

Suggested pairs

1. Function points plus analogous: one starts from your own design, the other from the market. The examples below use this pair.
2. Story points plus analogous: one starts from your backlog, the other from the market.
3. Function points plus story points: one starts from system features, the other from team judgment of story size.

Our two methods: 1. Function Points 2. Use-Case Points

Why these two use different starting information: Function Points estimate the system based on what users enter, view, and what data the system stores, while Use-Case Points estimate the system based on the actors who interact with it and the use cases they perform.

Step 3. Estimate the slice

Complete only the two sections (3A to 3D) that match your methods. Whatever method you use, finish with effort in team hours for your slice, so the two estimates can be compared in Step 5. Where you must choose a number, write it down and label it an assumption.

3A. Function points (Part 2, Slides 13 to 15)

What it measures: the functionality users get from the slice, counted in five categories. Your Step 1 table already lists most of what you need.

Category	What to count	Example from the permit slice
External Inputs (EI)	Forms where data enters the system	Register vehicle, purchase permit, update vehicle
External Outputs (EO)	Results the system calculates and sends	Permit confirmation and receipt
External Inquiries (EQ)	Simple lookups of stored data	Look up my permit, list lots with permits available
Internal Logical Files (ILF)	Groups of data the slice stores	Permit records, vehicle records
External Interface Files (EIF)	Data read from another system	Student registry, payment service

How to do it

1. Use your Step 1 table. Count each distinct input, output, lookup, and data group in the slice once.
2. Multiply each count by its weight and add the results. That total is your slice size in function points.
3. Choose a productivity rate in hours per function point. You have no history, so label it an assumption and test a low and a high value (Slide 15 shows 6 to 12 hours).
4. Multiply size by each rate to get a low and high effort in hours.

Category	Count	Weight	Points
External Inputs (EI)	3	4	12
External Outputs (EO)	1	5	5
External Inquiries (EQ)	3	4	12
Internal Logical Files (ILF)	2	10	20
External Interface Files (EIF)	0	7	0
Total function points			49

Counted items: each item in the count above, by name.

Category	Counted items	Screen
External Inputs (EI): 3	EI-1 Update location hours and open/closed status. EI-2 Add or edit menu items. EI-3 Mark an item sold out or back in stock (added October 5).	Staff update form

Category	Counted items	Screen
External Outputs (EO): 1	EO-1 Open-now status, calculated from stored hours and the current time.	Location list
External Inquiries (EQ): 3	EQ-1 List dining locations. EQ-2 Show a location's menu with each item's availability. EQ-3 Filter locations and items by dietary need.	Location list, menu view
Internal Logical Files (ILF): 2	ILF-1 Dining locations (name, hours, status). ILF-2 Menu items (location, dietary tags, availability, last updated).	None (stored data)
External Interface Files (EIF): 0	None. No outside system feeds the slice.	None

Rate: low 6 and high 12 hours per FP

Assumption or source: Based on the suggested 6–12 hours per function point range provided in the activity.

Effort: $49 \text{ FP} \times 6 \text{ h} = 294 \text{ hours (low)}$; $49 \text{ FP} \times 12 \text{ h} = 588 \text{ hours (high)}$

Example: 3 inputs (12), 1 output (5), 2 inquiries (8), 2 data groups (20), and 2 external systems (14) give 59 function points. At 6 to 12 hours per function point (assumed), effort is 354 to 708 hours; at 8 hours it is 472.

3B. Use-case points (Part 2, Slides 16 and 17)

What it measures: size based on who uses the slice (actors) and what they do with it (use cases). A use case is a goal a user completes, such as “purchase a permit.” Its steps are the back-and-forth exchanges between the user and the system.

How to do it

1. List your actors. A person using a screen is complex (3). Another system connected through a protocol is average (2). Another system connected through an API is simple (1). Add the weights to get UAW.
2. List the use cases in your slice and count the steps in each. 1 to 3 steps is simple (5), 4 to 7 is average (10), 8 or more is complex (15). Add the weights to get UUCW.
3. Add UAW and UUCW to get unadjusted use-case points (UUCP).
4. The technical factor (TCF) and environment factor (ECF) adjust for difficulty and team experience. If you do not score the full factor lists, use 1.0 for each and write that down as an assumption.
5. Multiply to get UCP, then multiply by a rate. Karner’s starting rate is 20 hours per UCP; label it an assumption.

Actors and Use Cases: The Student is a complex actor (3) because the student interacts with the system through a screen. Dining staff are also a complex actor (3) because they use the staff update form, so UAW = 6. The five student use cases are each simple (5), giving 25. The two staff use cases add 10 (average) and 5 (simple), so UUCW = 40. The mobile visibility story is treated as a usability requirement rather than a separate use case.

Actor or use case	Steps (user and system exchanges)	Weight
Student (actor)	Uses the location list and menu view: a person using a screen.	Complex, 3
Dining staff (actor, added October 5)	Uses the staff update form: a person using a screen.	Complex, 3
UC-1 View campus dining locations	2: student opens DineCheck; system lists the dining locations.	Simple, 5
UC-2 View hours and open status	2: student picks a location; system shows its hours and whether it is open.	Simple, 5
UC-3 View menu items	2: student opens the menu view; system lists that location’s menu items.	Simple, 5
UC-4 View item availability	2: student checks an item; system shows available or sold out and when it was last updated.	Simple, 5
UC-5 Filter by dietary need	3: student opens the filters; student picks restrictions or preferences; system shows the matching options.	Simple, 5
UC-6 Mark an item sold out (added)	4: staff signs in; system shows that location’s menu; staff marks an item sold out or back in stock; system saves it with the time and updates the menu view.	Average, 10
UC-7 Update hours and menu (added)	3: staff opens hours and menu; system shows the current entries; staff saves changes and the system confirms them.	Simple, 5

Assumptions: TCF and ECF are assumed to be 1.0 because the full factor lists were not scored. The rate is assumed to be 15 to 25 hours per UCP, with Karner’s starting rate of 20 hours in the middle, because we have no productivity data of our own. Effort: $46 \times 15 = 690$ hours (low); $46 \times 25 = 1,150$ hours (high); middle 920 hours.

Item	Calculation	Value
Actor weight (UAW)	Sum of actor weights	6
Use case weight (UUCW)	Sum of use case weights	40
Unadjusted (UUCP)	UAW + UUCW	46
Technical factor (TCF)	1.0 unless you scored it	1.0
Environment factor (ECF)	1.0 unless you scored it	1.0
Use-case points (UCP)	$UUCP \times TCF \times ECF$	46
Rate (hours per UCP)	20 unless you have a source	15–25
Effort (hours)	$UCP \times \text{rate}$	690–1,150

Example: the student is a complex actor (3); the registry and payment service connect by API (1 each), so UAW = 5. Register vehicle is simple (5), purchase permit has 8 or more steps (15), and look up permit is simple (5), so UUCW = 25. UCP = 30 with factors of 1.0 (assumed). At 20 hours each, effort is 600 hours.

3C. Story points to cost (Part 2, Slides 25 to 27)

What it measures: effort based on your team's own sizing of the stories in the slice. Story points are relative, so you convert them using a velocity (points per sprint). Because you have not run any sprints, velocity is an assumption. Use a low, middle, and high value.

How to do it

1. Use the total points from your Step 1 story table.
2. Choose a low, middle, and high velocity in points per two-week sprint. Label them assumptions.
3. Divide the slice points by each velocity. Keep the decimals; a small slice may take less than one sprint.
4. Multiply by team hours per sprint (team size $\times$ hours per person per sprint) to get effort in hours.

Input	Value	Note
Slice points		
Team size		
Hours per person per sprint		
Velocity: low / middle / high		
Sprints needed (high to low velocity)		
Effort in hours (low to high)		

Example: 21 points at 30, 20, or 15 points per sprint takes 0.7, 1.05, or 1.4 sprints. With 5 people working 80 hours each per sprint (400 team hours), effort is 280 to 560 hours.

3D. Analogous (Part 2, Slide 10)

What it measures: effort based on how long a similar real product took. You turn the product's timeline into person-months, scale it down to the size of your slice, then convert it to hours.

How to do it

1. Pick the similar product closest to your project from your Part 1 research, and note its team size and time to first release.
2. Multiply team size by months to get person-months. This is the effort the comparable product took.
3. Estimate what fraction of their first release your slice matches. If your slice is about a tenth of what they shipped, multiply by 0.1. Give a low and high fraction and explain them.
4. Adjust for team size. A much larger team spent more of its time coordinating, so its effort per feature is higher than yours will be.
5. Convert to hours. State the hours per person-month you assume (for example, 160).

Question	Answer
Comparable product, team size, and time to first release	
Person-months (team size × months)	
Fraction of their release our slice matches (low to high), and why	
Team size adjustment and why	
Adjusted effort in hours (low to high)	
Confidence (low, medium, high) and why	

Example: ParkEasy (fictional) took 4 people about 9 months, or 36 person-months, to launch a map of lots and pay-by-phone in one city. Buying a permit is roughly 8 to 12 percent of that release. 36×0.08 to $0.12 = 2.9$ to 4.3 person-months, or about 460 to 690 hours at 160 hours per person-month (about 576 hours at 10 percent).

Before you move on: put both estimates in hours

Step 5 compares your two estimates directly, so both must be in team hours for the same slice. If an estimate came out in months, sprints, or dollars, convert it now and show the conversion.

Step 4. Check your estimates for team size (Part 2, Slides 19 to 24)

Larger teams spend more of their time coordinating: meetings, handoffs, and fitting work back together. Estimators tend to underweight this cost, so check each estimate against the list below. Anything marked No needs a fix or a written explanation.

Check	Yes / No	Why it matters
Our team is five or six people at most.	yes	Beyond six, split into two teams.
We counted our communication paths: $n(n - 1) \div 2 = 6$	yes	Five people share 10 paths; ten share 45.
We added explicit work for integrating the pieces of the slice.	yes	Breaking work into pieces hides the cost of putting it back together.
We did not assume that more people means proportionally more output.	yes	Coordination grows faster than headcount.
We estimated for the team we have.	yes	Not for a larger team we wish we had.
We adjusted, not copied, timelines from much larger teams.	N/A	Their effort includes their coordination costs.

Integration work: about 87 of the 694 expected hours are testing and integration (the Testing role in our one-page budget), such as connecting the staff update form to the menu view and checking that a sold-out mark reaches students. These hours are inside the estimate, not added on top.

Larger-team timelines (N/A): we used function points and use-case points, not an analogous estimate, so no larger team's timeline went into these numbers.

Step 5. Compare your two estimates (Part 2, Slides 31 and 32)

Two estimates almost never match. The gap is useful: it points to the assumption you are least sure about. Your job is to find that assumption and name it.

	Method 1	Method 2
Method	Function Points	Use-Case Points
Estimate in hours (low to high)	294–588 hours	690–1,150 hours
Middle value in hours	441 hours	920 hours
Input that drives it most	Productivity rate (hours per FP)	Rate (hours per UCP)
Is that input measured or assumed?	Assumed	Assumed

For Method 1, the middle value is: $(294 + 588) \div 2 = 441$ hours

For Method 2, the middle value is: $(690 + 1,150) \div 2 = 920$ hours, the same as $46 \text{ UCP} \times 20 \text{ hours}$.

Gap $920 \div 441 = 2.09$

Questions to answer

1. **Do your two methods depend on the same assumption?** No. Function Points depend on the system's inputs, outputs, inquiries, stored data, and productivity rate, while Use-Case Points depend on the actors, use-case complexity, and hours per UCP.
2. **Which single input moves the answer most? Try changing it and see.** The Function Point productivity rate moves the answer the most. At 6 hours per FP, the estimate is 294 hours, while at 12 hours per FP, it increases to 588 hours. For use-case points, moving from 15 to 25 hours per UCP takes the estimate from 690 to 1,150 hours.
3. **What information would narrow the gap, and when could you get it?** Actual team productivity data would narrow the gap. Once development begins, we could measure the actual hours needed to complete features and use cases instead of relying on assumed productivity rates.

Our explanation of the gap (one paragraph)

The Function Point method produced an estimate of 294–588 hours with a middle value of 441 hours, while the Use-Case Point method produced 690–1,150 hours with a middle value of 920 hours. The gap between the middle values is about 2.09; it was 1.38 before we added the dining-staff actor. Adding the staff side added one input to the function points (+4 FP) but an actor and two use cases to the use-case points (+18 UCP), and use-case points give every use case at least 5 points even when it is a small lookup. Both methods also rely on assumed productivity rates because we do not yet have actual development data. Collecting real productivity data during development would help reduce this uncertainty.

Example: function points gave a middle value of 472 hours and the analogous method gave 576 hours, a gap of about 1.2. The two are fairly close, and they start from different information, so the agreement is meaningful. The largest uncertainty is the productivity rate, which is our assumption, and the fraction of ParkEasy's release that our slice represents.

Step 6. Present the estimate (Part 2, Slide 33)

This is the table your sponsor reads. It gives a range instead of a single number, shows what the range depends on, and is dated because estimates change as you learn more. It becomes the Estimation Appendix on your Sprint 2 portal page.

How to fill it in

1. Range: the lowest and highest hours across both methods.
2. Expected value: set O to your low end and P to your high end. Choose M, your most likely value, somewhere between them and explain why. Then calculate $(O + 4M + P) \div 6$.
3. Cost: multiply hours by a loaded hourly rate. State the rate and where it came from, such as published wage data plus an overhead percentage you choose.
4. Calendar time: divide hours by (team size $\times$ productive hours per person per week). State the productive hours you assume.

Field	Our estimate
Slice estimated	Student visibility of campus dining options, locations, menu availability/sold-out items, dietary filtering, and mobile visibility, plus the staff update form.
Range in hours (low to high)	294–1,150 hours
Expected hours: $(O + 4M + P) \div 6$, and why we chose M	694 hours. O = 294, M = 680, P = 1,150. $(294 + 4(680) + 1,150) \div 6 = 694$ hours. We chose M = 680 because it sits halfway between the two methods' middle values: $(441 + 920) \div 2 = 680.5$, rounded down. Both rates are assumed, so we had no evidence to favor one method.
Cost range and loaded rate used	\$17,640–\$69,000 (expected \$41,640) using a loaded rate of \$60/hour, the instructor-provided rate for CS 4390/5388.
Calendar estimate in weeks, and hours per week assumed	2.5–9.6 weeks. Assumes 4 team members $\times$ 30 productive hours/person/week = 120 team hours/week. Expected time $\approx$ 5.8 weeks.
Methods used	Function Points and Use-Case Points
Key assumptions	6–12 hours per FP; 15–25 hours per UCP; TCF = 1.0; ECF = 1.0; \$60/hour loaded rate; 30 productive hours/person/week; about 87 hours of testing and integration inside the estimate.
Team size assumed	4 people
What would change this estimate	Changes in project scope, additional features or integrations, actual team productivity, use-case complexity, and changes to dining/menu data requirements.
Date of this estimate	October 5, 2026 (revised; first completed September 28, 2026)

Example, “Buy a semester permit” slice: range 354 to 708 hours. With M = 520, expected hours = $(354 + 2,080 + 708) \div 6 \approx 524$. At an assumed loaded rate of \$60 per hour, cost is about \$21,000 to \$42,500. With 5 people at an assumed 30 productive hours a week (150 team hours), the expected 524 hours is about 3.5 weeks.

Before you submit

1. Your slice is named, with its stories, screens, data, and boundary written down.
2. Both estimates are for the same slice and in hours, with the conversions shown.
3. Every assumed number is labeled as an assumption.
4. Every sourced number has a source you can link.
5. The team-size check is complete, and any No has a fix or explanation.
6. The final estimate is a dated range, not a single number.
7. The individual estimation memo is your own work, written without AI.